



AI Fitness Trainer — Complete Technical Documentation

Real-time pose detection · automatic rep counting · live form coaching

A software-architecture & AI-engineering handbook
Frontend (Next.js) · On-device AI (TensorFlow.js) · PostgreSQL

Version: forge-ai-1.1 | Generated for the project creator

Table of Contents

1. Project Overview
2. Overall System Architecture
3. Folder Structure
4. Frontend
5. Backend
6. AI Pipeline (the big chapter)
7. Pose Detection & Landmark Map
8. BlazePose 3D
9. One-Euro Filter
10. Rep Counting Algorithm
11. Form Analysis
12. Exercise Recognition
13. Camera Logic
14. Computer Vision Foundations
15. Algorithms Used (catalog)
16. AI Models
17. Performance Optimization
18. Debug System (Developer Mode)
19. Security & Privacy
20. Deployment
21. 100 Interview Questions & Answers
22. 100 Viva Questions & Answers
23. Future Improvements
24. Final Summary

1. Project Overview

What is AI-Trainer (FORGE)?

FORGE is a web-based **AI personal fitness trainer**. The user opens their phone or laptop camera, picks an exercise (e.g. squat), and the app watches them through the camera in real time. It detects the body's joints, counts repetitions automatically, scores the quality of each rep, and speaks coaching cues out loud — exactly like a human trainer standing next to you. It also records the workout, tracks weights and personal records, and saves history.

What problem does it solve?

- **Cost & access:** Personal trainers are expensive and not always available. FORGE gives 24/7 AI coaching for the price of an app.
- **Bad form → injury:** Most people training alone don't know if their form is correct. FORGE detects mistakes (shallow depth, knees caving, rounded back) and corrects them live.
- **Motivation & tracking:** It counts reps so you don't have to, tracks progressive overload, detects PRs, and keeps you engaged with voice + a session report.

Why build it?

To combine three things that normally require a coach + a notebook + a video review: (1) real-time technique correction, (2) automatic logging, and (3) progress analytics — all running **on the user's own device** so it is cheap to operate and private by design.

Target audience

- Home / gym lifters who train alone and want form feedback.
- Beginners who don't know correct technique.
- Intermediate/advanced lifters who want automatic rep + volume + PR tracking.
- Initial market: India (price-sensitive, mobile-first), then global.

Real-world use cases

- Doing squats at home with the phone on a tripod — the AI counts and says "go lower".
- Tracking bench-press weight per set and getting a "new personal record" announcement.
- Recording a clean set to share on Instagram/WhatsApp with burned-in rep overlay.
- Reviewing a session report: form 88%, depth 90%, stability 75%, "your depth dropped near the end".

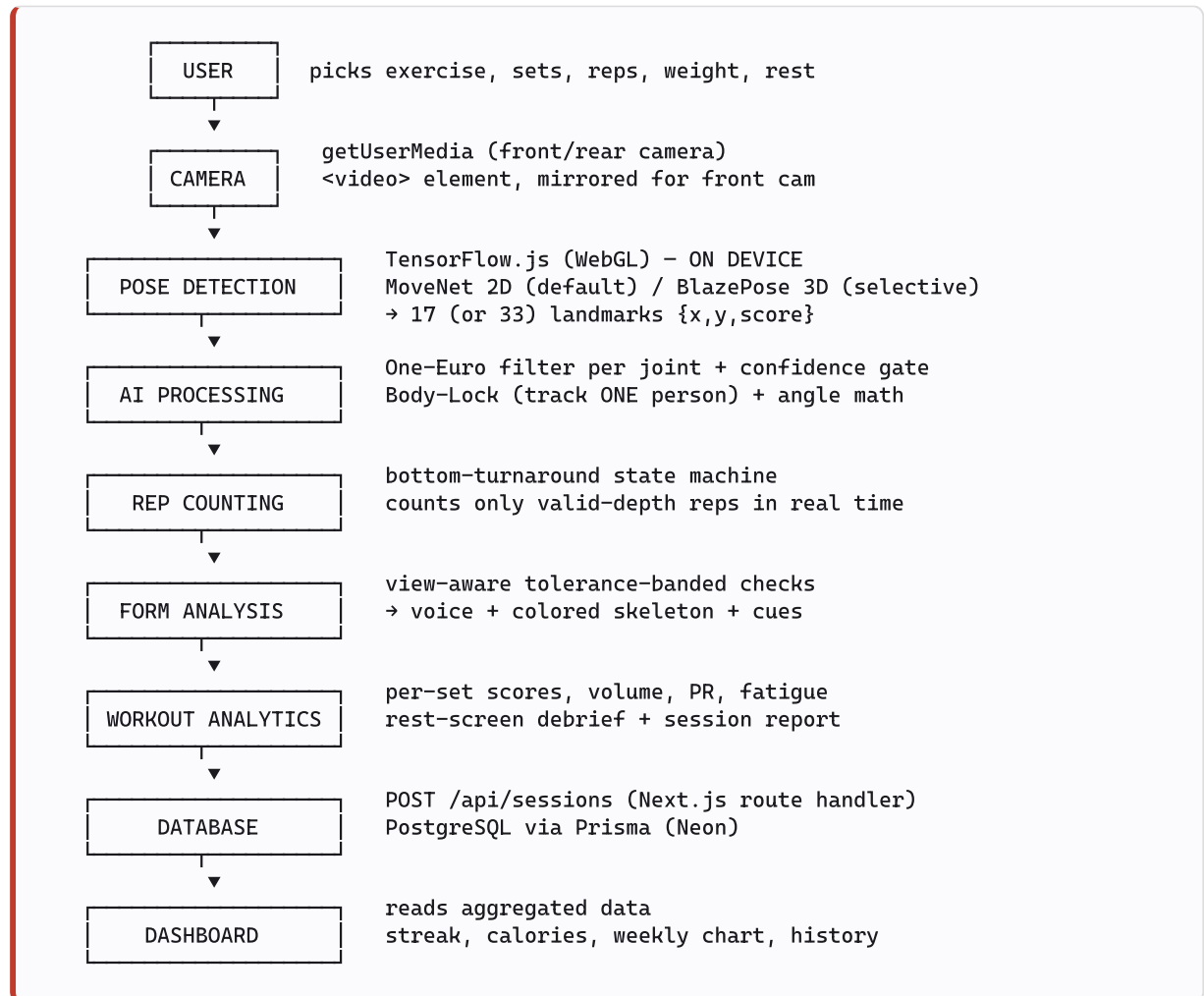
Future roadmap

AI workout recommendations, personalized plans, fatigue/injury-risk detection, LLM-powered conversational coaching, wearable/smartwatch integration, and a native Android app (true gallery save + on-device voice in recordings). See Chapter 23.

2. Overall System Architecture

FORGE is a **client-heavy** application: the expensive AI computation (pose detection, rep logic, voice) runs **in the browser on the user's device**. The server only handles auth, saving sessions, and serving data. This is the single most important architectural decision — it gives ~90% gross margins and strong privacy.

End-to-end pipeline



Why each connection exists

Edge	What flows	Why
User → Camera	A user gesture ("Start") opens the camera and unlocks audio	Browsers require a gesture for camera + speech (autoplay policy)
Camera → Pose	Raw video frames	The model needs pixels to estimate joint positions
Pose → AI Processing	Landmarks (x,y,score)	Smoothing + lock + angles turn noisy points into stable signals

AI → Rep Counting	Smoothed joint angles of the locked person	Reps are derived from one joint angle crossing thresholds
Rep → Form	Per-frame pose + per-rep aggregates	Form must be judged while the rep happens
Form → Analytics	Scores, faults, weights	To produce the rest debrief + final report
Analytics → DB	Session + per-set rows (JSON over HTTPS)	Persistence enables history, PRs, progressive overload
DB → Dashboard	Aggregated queries	Show streak, volume, charts

Three planes

Plane	Runs where	Tech
Presentation + AI	Browser (client)	Next.js (React 19), TensorFlow.js, Web Speech, Web Audio, MediaRecorder
Application/API	Server (Vercel/Render)	Next.js Route Handlers + Server Components, JWT auth
Data	Managed cloud	PostgreSQL (Neon) via Prisma ORM

3. Folder Structure

The project is a single Next.js App-Router codebase. Key folders:

```
AI-Trainer/
├── prisma/
│   ├── schema.prisma      # DB models (User, Profile, Exercise, WorkoutSession...)
│   ├── seed.ts            # loads exercises + equipment + achievements
│   └── data/
│       ├── exercises.ts   # 120 exercises (muscles, cues, poseKey, AI flags)
│       └── equipment.ts   # 34 equipment/machine catalog entries
├── src/
│   ├── app/               # routes (App Router)
│   │   ├── (app)/         # authenticated shell (sidebar layout)
│   │   │   ├── dashboard/ # command-center dashboard
│   │   │   ├── exercises/ # library list + [slug] detail
│   │   │   ├── workouts/  # custom workout builder + saved
│   │   │   ├── progress/  # profile/ settings/ admin/
│   │   │   └── settings/developer/ # HIDDEN dev tools
│   │   │       └── history/ # HIDDEN session-history dashboard
│   │   ├── train/[slug]/  # FULL-SCREEN AI trainer (no chrome)
│   │   ├── api/           # route handlers (auth, sessions, workouts, onboarding)
│   │   ├── login/ signup/ onboarding/ ...
│   │   ├── layout.tsx     # root layout (fonts, Toaster, SW cleanup)
│   │   └── globals.css    # design system (Tailwind v4 theme)
│   ├── components/
│   │   ├── trainer/       # the AI trainer feature (see below)
│   │   ├── ui/            # button, input, switch, progress-ring...
│   │   └── app/ auth/ dashboard/ exercises/ workouts/ onboarding/
│   ├── lib/
│   │   ├── pose/          # ALL the AI logic (see below)
│   │   ├── auth.ts jwt.ts prisma.ts validators.ts fitness.ts
│   │   ├── camera.ts weight.ts coach.ts voice.ts dev.ts dev-history.ts
│   │   ├── proxy.ts       # route protection (Next 16 middleware)
│   └── next.config.ts     # mediapipe alias, webpack/turbopack
```

The AI core — `src/lib/pose/`

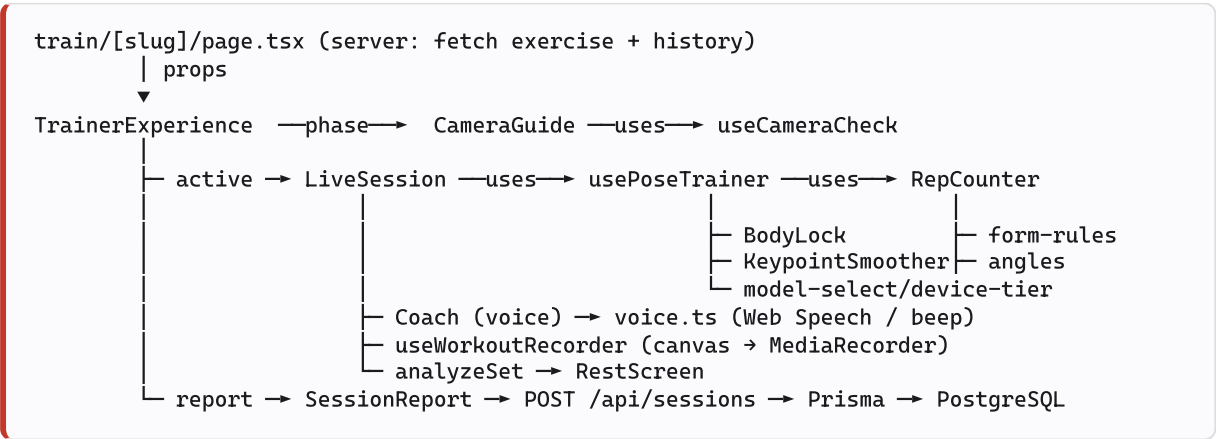
File	Responsibility
<code>angles.ts</code>	Geometry: keypoint map, joint angle, angle-from-vertical, confidence/best-side helpers
<code>one-euro.ts</code>	One-Euro adaptive low-pass filter + <code>KeypointSmoother</code> (per-joint, occlusion hold)
<code>exercises.ts</code>	Per-exercise rep config (joint triple, start/active/ideal angles, cues)
<code>form-rules.ts</code>	View-aware, tolerance-banded form checks + orientation detector
<code>rep-counter.ts</code>	The brain: turnaround state machine, ROM/form/stability scoring, attempts log
<code>set-analysis.ts</code>	Per-set debrief (mistakes, suggestions, fatigue, comparison)
<code>body-lock.ts</code>	Lock onto one person; re-identify across frames (anti person-switch)
<code>camera-setup.ts</code>	Per-exercise camera recommendations + required joints
<code>model-select.ts</code>	Hybrid 2D/3D choice (exercise × device tier × override)

device-tier.ts	Low/mid/high device classification
smoothing-config.ts	Tunable One-Euro params (dev settings)

The trainer UI — [src/components/trainer/](#)

trainer-experience.tsx	Phase machine: setup → camera guide → live → report
use-pose-trainer.ts	The live hook: loads model, runs the rAF loop, smoothing, lock, draw, telemetry
use-camera-check.ts	Camera-guide validation hook (visibility/angle/lighting/confidence)
camera-guide.tsx	Pre-workout camera setup + auto-start countdown
live-session.tsx	The full-screen HUD: counter, cues, controls, rest, recording, dev HUD
rest-screen.tsx	Between-sets debrief + next-set weight selector
session-report.tsx	Final scores, video, PR, save + export
use-workout-recorder.ts	Composite-canvas recording (MediaRecorder)

How files communicate



4. Frontend

Framework — Next.js 16 + React 19 + TypeScript

What it is: Next.js is a React framework that gives routing, server rendering, and an API layer in one project. React builds the UI from reusable components. TypeScript adds types so bugs are caught before running.

Choice	Why chosen	Alternatives & trade-off
Next.js (App Router)	One codebase for UI + API + auth; server components keep secrets server-side; fast routing; great deploy story	Plain React + Vite (no backend), Remix (smaller ecosystem), Angular (heavier)
React 19	Component model fits a stateful camera UI; huge ecosystem (TF.js, Framer Motion)	Vue/Svelte — fine, but less AI/ML web tooling
TypeScript	Pose data is shaped data ({x,y,score}); types prevent whole classes of bugs	Plain JS — faster to write, riskier
Tailwind CSS v4	Fast, consistent design system; custom dark-luxury theme in <code>globals.css</code>	CSS Modules, styled-components — more boilerplate
Framer Motion	Premium animations (counters, transitions) with little code	CSS animations — less control
Recharts	Declarative charts for progress/dashboard	Chart.js, D3 (D3 = more power, more code)

Routing

File-based: a folder under `src/app` = a URL. Two groups:

- `(app)/` — a **route group** (parentheses = not in the URL). All pages here share the authenticated sidebar layout: dashboard, exercises, workouts, progress, profile, settings, admin.
- `train/[slug]/` — the **full-screen trainer**, deliberately *outside* the app shell so there is no sidebar/chrome — only camera + overlays (a hard requirement of the spec). `[slug]` is a dynamic segment, e.g. `/train/barbell-squat`.

Route protection lives in `src/proxy.ts` (Next 16's middleware equivalent): it checks the JWT cookie and redirects unauthenticated users to `/login` before any protected page renders.

Server vs Client Components

By default Next renders components **on the server** (fast, secure, no JS shipped). Components that need browser APIs or interactivity opt in with `"use client"`. Example: `train/[slug]/page.tsx` is a *server* component that fetches the exercise + history from the DB, then passes them as props to `<TrainerExperience>`, a *client* component that runs the camera/AI.

Components

Organized by feature in `src/components/` (trainer, ui, dashboard, exercises, workouts, auth, onboarding). The `ui/` folder holds primitives (Button with `asChild` Slot pattern, Input, Switch, ProgressRing) reused everywhere for consistency.

State management

Deliberately **no Redux/Zustand**. State is local and layered:

- **React state/refs** inside the trainer hook for per-frame data (refs avoid re-rendering 30×/sec — critical for performance).
- **Server is the source of truth** for persistent data; pages fetch fresh.
- **localStorage** for device-local preferences and dev telemetry (smoothing params, engine override, dev session history).

Why refs, not state, in the loop: updating React state 30–60 times a second would re-render the whole tree and tank FPS. The AI loop writes to `useRef` values and draws to a canvas directly; only user-visible numbers (rep count, cue) are pushed to state, and the dev HUD is throttled to every 200 ms.

Camera integration

`navigator.mediaDevices.getUserMedia()` opens the camera into a hidden `<video>` element. A `<canvas>` sits on top to draw the skeleton + overlays. Front camera is **mirrored** (CSS + coordinate flip) so it feels like a mirror; rear camera is not. `facingMode` toggles front/rear (Chapter 13).

UI flow

```
Login → Onboarding (goals, level, equipment) → Dashboard
└─ pick Exercise → /train/[slug]
    Setup (sets/ reps/ weight/ rest/ mode) → Camera Guide (auto-start)
        → Live Session (reps + form + voice) ⇌ Rest Screen (debrief)
            → Session Report (scores, video, PR) → save → Dashboard
```

Performance optimization (frontend)

- Canvas drawing instead of DOM for the skeleton (GPU-friendly, no layout thrash).
- Refs + throttled HUD to avoid render storms.
- Model loaded once and reused; WebGL backend for GPU inference.
- Dynamic import of the heavy TF.js trainer so the rest of the app stays light.
- A `ServiceWorkerCleanup` component prevents stale cached bundles (a real bug we hit).

5. Backend

API architecture

The backend is a set of **Route Handlers** under `src/app/api/` — small server functions that respond to HTTP requests. They validate input with **zod**, talk to the DB through **Prisma**, and return JSON. There's no separate server process; Next.js runs the API and the UI together.

Endpoint	Method	Purpose
<code>/api/auth/signup</code>	POST	Create user (bcrypt-hash password), issue JWT cookie
<code>/api/auth/login</code>	POST	Verify credentials, set HTTP-only JWT cookie
<code>/api/auth/logout</code>	POST	Clear the auth cookie
<code>/api/auth/google</code>	POST	Verify Google OAuth token, find/create user, issue JWT
<code>/api/auth/me</code>	GET	Return the current user from the cookie
<code>/api/onboarding</code>	POST	Save profile (goals, level, equipment, metrics)
<code>/api/sessions</code>	POST/GET	Save a finished workout (sets, reps, scores, volume); list history
<code>/api/workouts</code>	POST/GET	Create/list custom workouts from the builder
<code>/api/exercises</code>	GET	Query the exercise library (filters)

Authentication

Custom JWT auth (no third-party auth service):

- **Signup/Login:** password hashed with **bcryptjs** (slow on purpose, salted) and stored — never the plaintext.
- **Token:** a signed **JWT** (via `jose`) put in an **HTTP-only, Secure, SameSite cookie** — JavaScript can't read it, so XSS can't steal it.
- **Google OAuth:** `google-auth-library` verifies the Google ID token server-side, then we issue our own JWT.
- **Protection:** `src/proxy.ts` guards every `(app)` and `train` route — mandatory login, as required.

Choice	Why	Alternative
Custom JWT	Full control, no per-MAU cost, simple	Auth0/Clerk (faster but paid + lock-in), NextAuth (heavier)
<code>jose</code>	Modern, works in Edge runtime	<code>jsonwebtoken</code> (Node-only)
HTTP-only cookie	XSS-safe token storage	<code>localStorage</code> token (XSS-vulnerable)

Database

PostgreSQL (hosted on **Neon**, serverless, pooled connection) accessed via the **Prisma** ORM. Prisma gives a typed schema and migrations; you write `prisma.user.findUnique( ... )` instead of raw SQL, and the types flow into the app.

Core models (in `prisma/schema.prisma`): `User`, `Profile`, `Exercise`, `Equipment`, `WorkoutSession` + `SetEntry`, `CustomWorkout`, `Achievement`. A workout session stores per-set reps, weight, scores, and volume so the dashboard and progress charts can aggregate it.

Choice	Why	Alternative
PostgreSQL	Relational data (users→sessions→sets), reliable, free tier	MongoDB (less fit for relations), MySQL (fine)
Neon	Serverless, scales to zero, pooled — cheap for a new app	Supabase, Railway, RDS
Prisma	Type-safe queries, migrations, great DX	Drizzle (lighter), raw SQL (error-prone)

Storage

Workout **videos stay on the device** (recorded via MediaRecorder, saved via download / Web Share) — no media upload cost or storage liability. Only structured session data goes to PostgreSQL. Dev telemetry lives in `localStorage`.

Logging & error handling

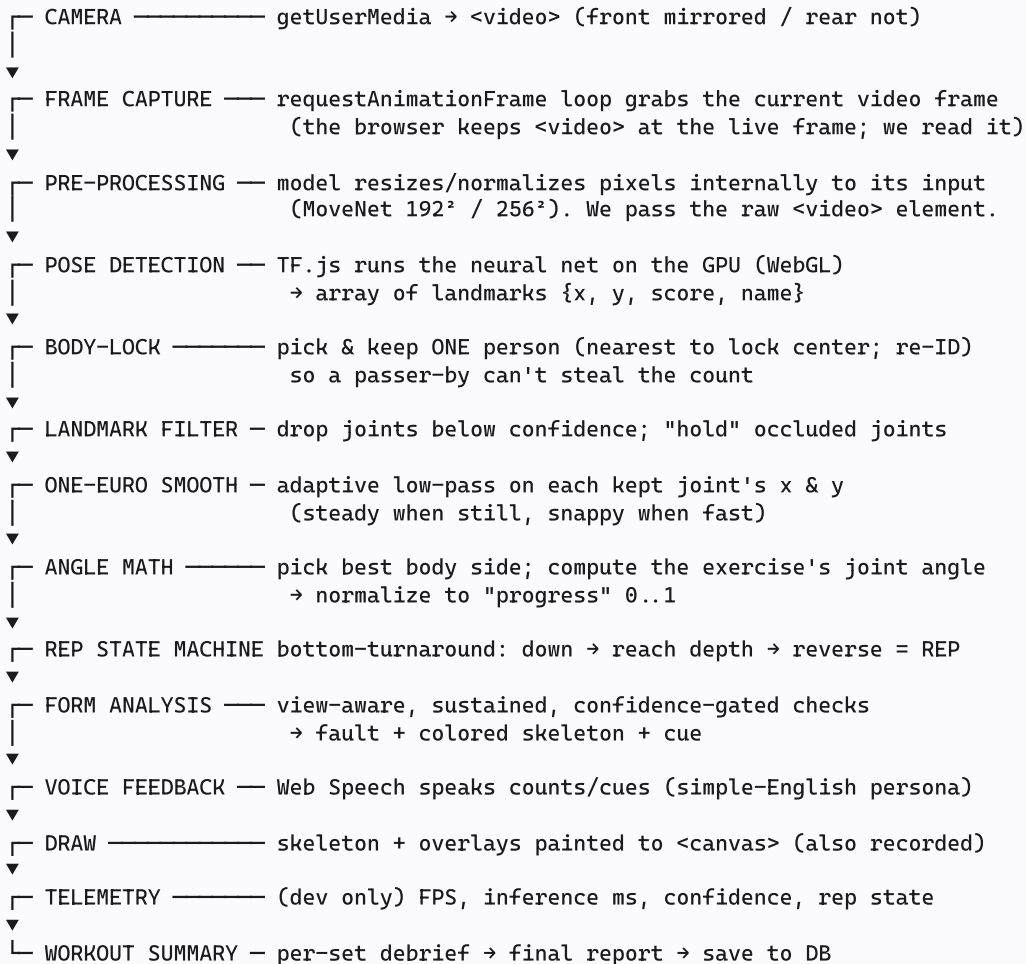
- API handlers wrap logic in try/catch and return proper HTTP codes (400 validation, 401 auth, 500 server) with JSON messages.
- zod validates every request body; invalid input is rejected before touching the DB.
- Client shows user-friendly toasts; the AI loop has its own error states (no camera, model load failed, tracking lost).
- Dev Mode adds a structured in-memory debug log (capped at 8000 events) exportable as JSON/CSV.

Deployment

Hosted on **Render** (and Vercel-compatible). Code on GitHub; push to `main` auto-deploys. Build = `prisma generate` + `next build`. Secrets (`DATABASE_URL`, JWT secret, Google client id) are environment variables, never committed. See Chapter 20.

6. AI Pipeline (the big chapter)

This chapter follows a single video frame from the camera all the way to a counted rep and a spoken cue. The whole pipeline runs **on the user's device, ~30 times a second**, inside the browser. Most of it lives in `use-pose-trainer.ts` (the loop) and `src/lib/pose/` (the math).



Stage-by-stage

1. **Camera:** A user gesture starts the stream (required by browsers) and also "unlocks" speech audio. Front camera is mirrored so it feels like a mirror.
2. **Frame capture:** We use `requestAnimationFrame` (rAF) — the browser's "draw a frame now" callback. Each tick we read the current `<video>` frame. rAF auto-throttles to the display refresh and pauses on hidden tabs (battery friendly).
3. **Pre-processing:** The detector internally resizes the frame to its fixed input (e.g. 256×256), scales pixel values, and produces a tensor. We don't hand-code this — the model does it.
4. **Pose detection:** The CNN outputs joint positions + confidence. 2D model (MoveNet) → 17 joints; 3D model (BlazePose) → 33 joints with depth. Runs on the GPU via WebGL.
5. **Body-lock:** With multi-person output, we select one person (the one near the lock point / largest+centered) and re-identify them each frame so the rep count stays glued to the user.
6. **Landmark filtering:** Joints below a confidence threshold are ignored; briefly occluded joints are "held" at their last good spot to avoid a wild jump.

7. **One-Euro smoothing:** Each kept joint's x and y are passed through a One-Euro filter to kill jitter without adding lag (Chapter 9).
8. **Angle math:** Choose the more visible body side, compute the exercise's defining angle (e.g. hip-knee-ankle for a squat), and normalize it into a 0→1 "progress" value (Chapter 10).
9. **Rep state machine:** A turnaround detector counts a rep the instant the user reverses out of a deep-enough bottom — in real time, no pause needed.
10. **Form analysis:** Form rules run while descending; a fault must persist several frames and clear a confidence bar before it speaks or rejects a rep.
11. **Voice:** Counts and cues are spoken via the Web Speech API with a friendly, simple-English persona; a beep fallback exists.
12. **Draw + telemetry:** The skeleton/overlays are drawn to a canvas (also the recording source). In Dev Mode a HUD shows FPS/inference/confidence.
13. **Summary:** At set end we compute scores; the rest screen debriefs; the final report saves to the DB.

Why on-device? Streaming video to a server for inference would add 100–300 ms latency (too slow for live coaching), cost a lot of GPU/bandwidth, and create privacy/legal exposure. On-device = instant, cheap, private.

7. Pose Detection & Landmark Map

Which model & why

The default detector is **MoveNet MULTIPOSE_LIGHTNING** (TensorFlow.js). "MultiPose" = detects several people; "Lightning" = the fast variant. Selectively we switch to **BlazePose** for 3D (Chapter 8).

Why MoveNet	Detail
Real-time on mid-range phones	30–60 FPS on WebGL — essential for live coaching
Multi-person + tracking	Needed for body-lock in a busy gym
Runs fully in-browser	No server, private, cheap
Good accuracy/speed trade-off	Lightning is fast; Thunder is more accurate but slower

How it works internally (plain English)

MoveNet is a **convolutional neural network (CNN)**. Convolutions slide small filters across the image to detect edges → shapes → body parts. The network outputs a **heatmap** per joint — a grid where the brightest cell is where that joint most likely is. The peak of each heatmap becomes the joint's (x, y), and the peak's strength becomes the **confidence score (0–1)**. Coordinates are normalized to the image, then scaled to canvas pixels.

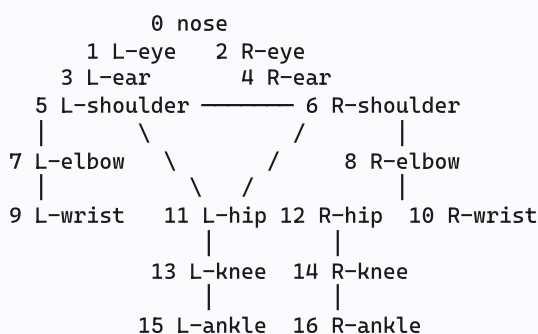
Confidence scores

Every joint has a score. We use thresholds throughout: <0.25 → occlusion-hold (Chapter 9); <0.3 (`MIN_CONF`) → joint unusable for rep math; <0.5 (`HIGH_CONF`) → a detected fault is demoted to a warning rather than rejecting a rep. This prevents the AI from "shouting" about mistakes it isn't sure it saw.

Tracking

MoveNet's tracker assigns IDs across frames so a person stays the "same" person. On top of that we run our own **Body-Lock** (Chapter 13) for robustness.

The 17-landmark map (MoveNet / COCO)



#	Joint	Used for
0-4	nose, eyes, ears	Orientation (front/side), head position
5/6	shoulders	Press/curl/push-up angle, torso, stability
7/8	elbows	Curl, press, push-up (middle joint)
9/10	wrists	Curl/press end joint, "wrist above shoulder" gate
11/12	hips	Squat/deadlift angle, sway/stability, center-of-mass
13/14	knees	Squat/leg-press/leg-extension middle joint
15/16	ankles	Squat end joint, base of support

BlazePose adds 16 more (hands' fingers, feet's heels/toes, mid-face) for 33 total — useful for foot/hand orientation and depth.

8. BlazePose 3D

What it is

BlazePose is Google's pose model (the one behind MediaPipe). In FORGE it runs through TensorFlow.js (tfjs runtime, "lite" variant). It returns **33 landmarks**, each with x, y and **z (depth)** plus visibility — a full 3D skeleton from a single camera.

How it works

Two stages: a fast **detector** finds the person and a region of interest, then a **tracker/regressor** predicts all 33 keypoints (and an estimated z) for that region. By tracking the ROI frame-to-frame it avoids re-detecting every frame, which keeps it fast for one person.

2D vs 3D

	MoveNet (2D)	BlazePose (3D)
Landmarks	17 (x, y)	33 (x, y, z, visibility)
People	Multi-person	Single person
Depth	No	Yes (estimated)
Speed	Faster	Heavier
Best for	Most reps, busy scenes, body-lock	Depth-sensitive faults (back rounding, toward/away-from camera motion)

Why only some exercises use it

Depth only matters for certain faults. Whether a squat hits depth or a curl reaches the top is fully visible in 2D from the side. But "is the back rounding?" or movements that travel *toward/away* from the camera benefit from z. So `model-select.ts` uses 3D only where it pays off — and only on capable devices. Also, BlazePose is **single-person**, so in 3D mode body-lock degrades (a real trade-off). Production defaults to **Auto**, which is 2D for most cases.

Advantages / Limitations / Performance

- **Advantages:** depth-aware faults, richer 33-point skeleton, stable single-person tracking.
- **Limitations:** z is *estimated* from one camera (not true depth), single-person only, heavier compute.
- **Performance:** on weak devices it drops FPS, so a runtime guard (Chapter 17) downgrades 3D→2D if FPS sags below a floor for ~2.5 s. The static device-tier check is the first gate; the FPS fallback is the real guarantee.

9. One-Euro Filter

Raw pose landmarks **jitter** — even when you hold still, each joint wobbles a few pixels every frame because the model re-estimates from scratch. Jitter makes angles noisy, which causes false reps and flickering form colors. The **One-Euro filter** (Casiez et al., CHI 2012) removes jitter *without* the lag a normal smoother adds.

The core idea

It's an **adaptive low-pass filter**: smooth hard when the joint is nearly still (kills jitter), but smooth lightly when the joint moves fast (stay responsive, no lag). It adapts the cutoff frequency to the measured speed.

Algorithm & formulas

Smoothing factor (exponential filter):

$$\tau = 1 / (2\pi \cdot f_c) \quad \alpha = 1 / (1 + \tau/\Delta t)$$

- 1) Estimate speed: $dx = (x_t - x_{t-1}) / \Delta t \rightarrow$ smooth it with $\alpha(d_cutoff)$: $\hat{e}x$
- 2) Adapt cutoff: $f_c = \text{minCutoff} + \beta \cdot |\hat{e}x|$
- 3) Smooth value: $\hat{x}_t = \alpha(f_c) \cdot x_t + (1 - \alpha(f_c)) \cdot \hat{x}_{t-1}$

Because Δt comes from real timestamps, the filter is **FPS-independent** — it behaves the same at 24 or 60 FPS. This is the exact code in `one-euro.ts`.

Parameters (tunable in Dev Settings)

Param	Default	Meaning	Effect
<code>minCutoff</code>	1.7	Baseline smoothing when still	Lower = smoother but more lag; higher = snappier but more jitter
<code>beta</code> (β)	0.015	Speed reactivity	Higher = less lag on fast/heavy reps; lower = steadier but can lag
<code>dCutoff</code>	1.0	Smoothing of the speed estimate	Rarely changed; lower if it feels twitchy when accelerating

Why it improves rep counting

The rep state machine watches the joint angle reverse direction at the bottom. Jitter creates fake tiny reversals → double counts or phantom reps. One-Euro removes those micro-reversals while preserving the real turnaround, so counts are clean. It also steadies the form colors and ROM/stability scores.

Practical example

Holding the bottom of a squat: raw knee angle might read 98°, 101°, 97°, 100°... One-Euro outputs a steady ~99°, so no false "you came up" trigger. Then you explode upward fast: β raises the cutoff so the filter tracks the quick rise with almost no lag, and the rep registers immediately.

Occlusion hold: the `KeypointSmoother` also "holds" a joint at its last good position when confidence drops below 0.25 (e.g. a dumbbell briefly hides the wrist), preventing a smoothed-but-wrong jump.

10. Rep Counting Algorithm

Implemented in `rep-counter.ts` as a **bottom-turnaround (extremum) state machine**. A rep is counted the **instant the user reverses out of a deep-enough bottom** — no pause, no full lockout, no return-to-top needed. This was a deliberate redesign after gym testing showed the old method only counted when you stopped moving.

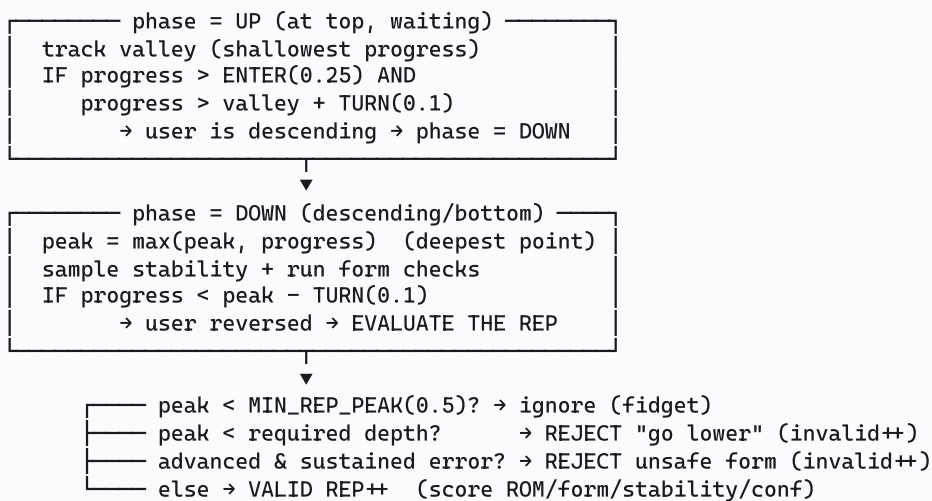
Step 1 — Angle → progress

Each exercise defines a joint triple and three angles (`exercises.ts`). We compute the live angle and normalize it:

$$\text{progress} = (\text{angle} - \text{startAngle}) / (\text{activeAngle} - \text{startAngle}) \rightarrow \text{clamped to } [0,1]$$

progress $\approx$ 0 at the resting/top position, $\approx$ 1 at the worked/bottom position. Example squat: start 160°, active 100°, ideal 70°.

Step 2 — The state machine



Step 3 — Thresholds (tuning constants)

Const	Value	Role
ENTER	0.25	Progress past which a descent can begin
TURN	0.1	Reversal needed to confirm a direction change (anti-jitter)
MIN_REP_PEAK	0.5	Minimum depth or it's a fidget, not a rep
MIN_CONF	0.3	Joint confidence floor to use it at all
HIGH_CONF	0.5	Confidence needed for an error to reject a rep
SUSTAIN	6 frames	A fault must persist this long to count

Step 4 — Mode-adjusted depth

Required depth eases by $\pm$ degrees per mode: Beginner gets a **+12°** easier target, Advanced a **-6°** stricter one. So beginners aren't punished for slightly shallow reps while advanced lifters are held to standard.

Edge cases handled

Case	How it's handled
Heavy weight / grind	Counted on turnaround, not lockout — a slow heavy rep still registers as soon as you reverse
Partial reps	Rejected if <code>peak < required depth</code> → "Go lower" cue, logged as invalid
Paused reps	Pausing at the bottom doesn't matter — the count triggers on the reversal whenever it happens
Fidgets / re-grip	Ignored via <code>MIN_REP_PEAK</code> and the TURN band
Too-fast reps	If <450 ms since last rep → "Slow down, control it" cue
Overhead press false reps	<code>requireWristAboveShoulder</code> : arms hanging (also a straight elbow) read as start, not a rep
Jitter double-count	TURN band + One-Euro smoothing + a light internal EMA (SMOOTH 0.35)
Tracking loss	If the joint drops below confidence, the frame is skipped (no phantom reps)

False positives / negatives — honest limits

The engine can't know "ground truth" without video review. **Missed reps** are proxied by the engine-rejected count; **false reps** can only be confirmed by watching the recording. The TURN band + min-depth + confidence gates minimize both, but they're a heuristic, not perfection.

Failure sets & counting modes

Two modes: **fixed** (target reps; the set auto-completes when hit) and **failure / reps-to-failure** (you press "Finish set" when you can't continue; the AI logs how many real reps you achieved). Both share the same counting core.

11. Form Analysis

Form checks live in `form-rules.ts` and are evaluated by the rep-counter during the DOWN phase (from `CHECK_FROM = 0.35` progress onward). The design goal: **never nag about a mistake the AI isn't confident it saw**.

The three gates before a fault "counts"

1. **View gate:** a check only runs if the camera view matches what it needs (e.g. a "back rounding" check needs a side view). `viewMatches(check.view, orientation)` .
2. **Confidence gate:** if the involved joints are below `HIGH_CONF (0.5)` , an "error" is **demoted to a warning** — visible but it won't reject a rep.
3. **Persistence gate:** a status must hold for `SUSTAIN = 6` consecutive frames (run-length tracking) before it's "active". One noisy frame never triggers a cue.

How a rep is judged Good / Bad / Incomplete

Verdict	Condition
Perfect	ROM ≥ 90 AND form ≥ 85 AND stability ≥ 80
Good (valid)	Reached required depth, no sustained high-confidence error (in advanced)
Incomplete	<code>peak < required depth</code> $\rightarrow$ rejected, "Go lower" type cue
Bad / cheating	Sustained high-confidence form error (advanced rejects it; beginner warns)

Scoring formulas

$$\text{form} = \max(0, 100 - \text{errors} \times 30 - \text{warns} \times 12)$$
$$\text{rom} = \text{clamp}(\text{peak} / \text{idealProgress}) \times 100 \quad (\text{idealProgress from idealAngle})$$
$$\text{stability} = \text{clamp}(1 - (\text{hipSwayRange} / \text{torsoLength}) \times 1.2) \times 100$$
$$\text{confidence} = \text{mean}(\text{per-frame min joint score}) \times 100$$

Detecting specific faults

Fault	How it's detected
Partial ROM / shallow	peak progress vs required depth
Swinging / instability	hip-center horizontal sway range $\div$ torso length (sampled each frame)
Knees caving, torso lean, etc.	Angle/posture checks vs tolerance bands, view-aware
Too fast (momentum/cheat)	time between reps < 450 ms
Plank/hold sag	shoulder-hip-knee angle vs posture threshold (hold mode)

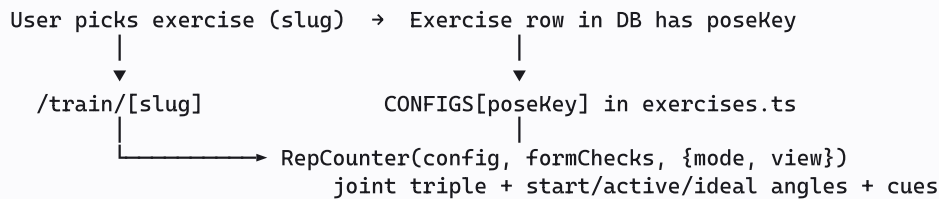
Faults map to a **colored skeleton**: green = good, amber = warning, red = error/danger, idle = at rest. The worst active fault drives the color and the joints it highlights.

Honest limit: true back-rounding and heel-lift need depth or a clean side view. In 2D from a bad angle these are reported conservatively (warning, not rejection) to avoid false alarms.

12. Exercise Recognition

Important honest point: FORGE does **not** auto-classify which exercise you're doing from raw video. It uses **declared-exercise + configuration matching**, which is the right engineering choice here. The user picks the exercise (e.g. from the library), and the trainer loads that exercise's tuning.

How "recognition" actually works



Each exercise's `poseKey` selects a `RepConfig` : which joints to measure, the start/active/ideal angles, the incomplete-rep cue, and special gates (e.g. `requireWristAboveShoulder` for presses). Families share configs — `squatLike` , `curlLike` , `pushupLike` , `pressLike` , `legExtensionLike` , `legPressLike` , etc.

Why declared, not classified

Declared (chosen)	Auto-classification (rejected for v1)
100% reliable — no misclassification	Easily confused (squat vs leg-press vs lunge look similar)
Zero extra latency/compute	Needs a second model running each frame
Lets us tune each exercise precisely	Hard to tune; needs a large labeled dataset
Matches real UX (you choose your workout)	Solves a problem users don't really have

Config "confidence" & switching

The relevant confidence is **landmark** confidence (per joint), which gates rep counting and form checks. Switching exercises = navigating to a different `/train/[slug]` , which instantiates a fresh `RepCounter` with that exercise's config. The orientation detector (`detectOrientation`) recognizes *camera view* (front/side), which the form checks adapt to.

Supported exercises

120 exercises in the catalog; the AI-tracked subset (≈ 77) have a `poseKey` + config. Categories: squats/lunges (knee-driven), presses (overhead/bench), curls/extensions (elbow), rows/pulls, push-ups/dips, leg machines (leg-press, leg-extension, leg-curl), and holds (plank, wall-sit). Non-tracked exercises still appear in the library with cues but skip live counting.

13. Camera Logic

Camera setup is the single biggest factor in tracking accuracy. `camera-setup.ts` stores per-exercise recommendations; `camera-guide.tsx` + `use-camera-check.ts` validate the setup before the workout and **auto-start** when the user is correctly positioned (no Start button).

Best positions per exercise

Exercise type	Best angle	Why
Squat, deadlift, lunge	Side-on	Depth + back angle are only visible from the side
Overhead/shoulder press	Front or side	Lockout visible both ways; front shows symmetry
Bicep curl	Front or side	Elbow angle clear; front shows both arms
Push-up, plank	Side-on, low	Body-line sag needs a side profile
Leg press / extension	Side-on	Knee angle range is a side-view measurement

What the camera guide checks

- **Body visibility:** are the required joints (from `camera-setup.ts`) all detected above threshold?
- **Distance/framing:** is the whole body in frame (not too close/far)?
- **Angle/orientation:** does the detected orientation match the exercise's required view?
- **Lighting/confidence:** is average landmark confidence high enough?

When all green for a short, sustained period, a countdown auto-starts the set.

Front vs rear camera

`camera.ts` handles `facingMode` (`user` = front, `environment` = rear) and a flip control. Front camera is **mirrored** (so it feels natural); the mirror is undone in coordinate math so left/right joints stay correct. Rear camera (on a tripod) is the recommended setup for lower-body work.

Occlusion, lighting, distance, height

Problem	Mitigation
Equipment hides a joint	Occlusion-hold in the smoother + confidence gating
Low light → low confidence	Guide warns; confidence gate prevents bad reps; cues to improve lighting
Too close/far	Guide checks full-body framing before start
Camera too high/low	Recommended waist-height + side-on in the guide

Multi-person handling & tracking lock (Body-Lock)

In a gym, others walk through frame. `body-lock.ts` locks onto one person and re-identifies them each frame (nearest to the lock center / largest-and-centered), so a passer-by can't hijack your rep count. The user

can also tap to set the lock target, and reset it. (In 3D/BlazePose mode, which is single-person, lock degrades — a known trade-off.)

14. Computer Vision Foundations

This chapter explains the CV concepts the app relies on — useful for "explain the fundamentals" questions.

Landmarks & coordinates

A **landmark** (keypoint) is a body joint the model locates: `{x, y, score, name}` (plus `z` in 3D). Coordinates are first normalized (0–1 of the image) then scaled to canvas pixels. Image y grows **downward** (top-left origin) — important when comparing "above/below" (e.g. wrist above shoulder means smaller y).

Vectors & joint angles

To measure a joint angle at B between A and C, form two vectors BA and BC and use the dot product:

$$\begin{aligned} \text{BA} &= \text{A} - \text{B}, \text{BC} = \text{C} - \text{B} \\ \cos \theta &= (\text{BA} \cdot \text{BC}) / (|\text{BA}| \cdot |\text{BC}|) \rightarrow \theta = \arccos(\cos \theta) \times 180/\pi \end{aligned}$$

This is exactly `angle(a,b,c)` in `angles.ts` (clamped to avoid NaN). `angleFromVertical` uses `atan2(dx, dy)` to measure a segment's tilt vs vertical (for torso lean / body-line checks).

Distance calculations

Euclidean distance `hypot(dx, dy)` measures limb/segment lengths — used for vector magnitudes, torso length (stability normalization), and framing checks.

Temporal smoothing & motion tracking

Per-joint One-Euro filtering over time (Chapter 9) plus a light EMA on the rep angle. Motion is tracked as the change in the normalized "progress" signal; its direction (increasing/decreasing) drives the rep phase.

Depth estimation

2D has no depth. BlazePose 3D estimates `z` from a single camera via a model trained on 3D-labeled data — an *inference*, not a measurement, so it's approximate. Used only for depth-sensitive faults (Chapter 8).

Best-side selection & orientation

`bestSide` averages confidence over each body side and uses whichever is more reliably seen (so a side-on user is tracked on the camera-facing side). `detectOrientation` classifies the camera view (front vs side) from the relative positions of shoulders/hips, letting form checks adapt.

15. Algorithms Used (catalog)

Every notable algorithm in the project, what it does, why it's needed, complexity, and trade-offs. n = landmarks per frame (17 or 33); per-frame work is effectively $O(n) \approx \text{constant}$, so real-time cost is dominated by the neural-net inference.

Algorithm	What / why	Big-O	Advantages	Limitations
Pose estimation (CNN)	MoveNet/BlazePose locate joints from pixels — the foundation	$O(1)/\text{frame}^*$	Real-time, on-device	Fixed cost; accuracy varies with view/light (*per fixed input size)
One-Euro filter	Adaptive low-pass to remove jitter without lag	$O(n)$	FPS-independent, tiny cost	Needs tuning per use-case
Rep state machine	Bottom-turnaround extremum detection → real-time reps	$O(1)/\text{frame}$	Counts mid-motion, robust to pauses	Threshold-based heuristic
Angle calculation	Dot-product interior angle at a joint	$O(1)$	Exact, cheap	2D only sees the projected angle
Joint vector math	Vectors, magnitudes (hypot), atan2 for tilt	$O(1)$	Standard geometry	Sensitive to noisy points (mitigated by smoothing)
Landmark smoothing (EMA)	Light exponential average on the rep angle	$O(1)$	Extra stability	Adds slight lag
Occlusion hold	Freeze a low-confidence joint at last good spot	$O(1)$	Prevents wild jumps	Stale if occlusion is long
Body-Lock tracking	Lock + re-identify one person across frames	$O(p)$	Anti person-switch in gyms	Degrades in single-person 3D mode (p = people)
Device-tier auto-fallback	Classify device + runtime FPS guard 3D→2D	$O(1)$	Keeps FPS above floor	Static check is approximate; FPS guard is the real guarantee
Hybrid 2D/3D selection	Pick engine by exercise × tier × override	$O(1)$	Depth only where it pays	3D loses multi-person
Confidence filtering	Threshold gates on landmark scores	$O(n)$	Suppresses false faults/ reps	May ignore real but low-confidence data
Exercise config matching	Declared exercise → RepConfig (not ML classification)	$O(1)$	100% reliable, tunable	User must pick exercise

Run-length form gating	Require SUSTAIN frames before a fault fires	O(checks)	Kills one-frame false alarms	Small reaction delay
Stability (sway) metric	Hip-sway range ÷ torso length	O(1)	Catches swinging/cheating	View-dependent
Orientation detection	Front vs side from joint geometry	O(1)	Makes form checks view-aware	Ambiguous at 45°
Frame sampling (RAF)	requestAnimationFrame drives the loop	—	Syncs to display, pauses hidden tabs	Coupled to refresh rate
Video encoding pipeline	canvas.captureStream → MediaRecorder (webm/mp4)	O(frames)	Burned-in overlays, on-device	Can't capture system TTS audio on web
Throttling/EMA telemetry	FPS/inference EMA, 200 ms HUD throttle	O(1)	Low overhead metrics	Dev-only

16. AI Models

Model 1 — MoveNet MULTIPOSE_LIGHTNING (primary, 2D)

Architecture	CNN (MobileNet-style feature extractor + center/heatmap heads), multi-person
Inputs	RGB video frame (resized to a fixed square internally)
Outputs	Up to 6 people × 17 keypoints {x, y, score} + tracking IDs
Strengths	Fast (30–60 FPS), multi-person, on-device, good speed/accuracy
Weaknesses	2D only (no depth), accuracy drops with occlusion/poor light/odd angles
Why chosen	Real-time coaching needs speed + multi-person for body-lock

Model 2 — BlazePose (selective, 3D)

Architecture	Two-stage detector + keypoint regressor (tfjs runtime, "lite")
Inputs	RGB frame + tracked region of interest
Outputs	33 keypoints {x, y, z, visibility}, single person
Strengths	Depth-aware, dense skeleton, stable single-person tracking
Weaknesses	Heavier, single-person, z is estimated not measured
Why chosen	Depth-sensitive faults on capable devices

Supporting "models"

Web Speech (speechSynthesis) — the browser's built-in TTS voice for coaching (not a model we ship; a platform API). A beep tone is the fallback.

Alternatives considered

Alternative	Why not (for v1)
MoveNet SinglePose Thunder	More accurate but single-person + slower — loses body-lock
MediaPipe Pose (native)	<code>@mediapipe/pose</code> had no clean ESM build (aliased to a stub); used BlazePose via tfjs instead
PoseNet	Older, less accurate than MoveNet
OpenPose	Server/GPU heavy — not browser-real-time
Server-side inference	Latency + cost + privacy — defeats the on-device thesis

17. Performance Optimization

Live coaching means everything per frame must finish in ~16–33 ms. Optimizations span the whole stack.

FPS & adaptive model selection

- **Static device-tier** (`device-tier.ts`): low/mid/high gates whether 3D is even attempted.
- **Runtime FPS guard** (in `use-pose-trainer.ts`): an FPS EMA; if FPS sits below a floor (24 mid / 18 low) for ~2.5 s, it **downgrades 3D→2D** automatically (Auto mode only) and logs a fallback event.
- This two-layer approach guarantees responsiveness even when the static guess is wrong.

CPU / GPU

- TF.js **WebGL backend** runs inference on the GPU; the CPU handles only light geometry.
- Model loaded **once** and reused (`modelRef`); no reload per set.
- Skeleton drawn on **canvas** (GPU compositing) instead of DOM nodes.

Memory

- Tensors are short-lived per frame; the detector manages disposal.
- Debug log capped at **8000 events**; dev session history capped at **200**.
- Refs instead of state arrays for per-frame data → no React garbage from re-renders.

Latency reduction

- On-device inference (no network round-trip).
- One-Euro tuned for low lag (β raises responsiveness on fast reps).
- Reps counted on turnaround (instant), not on lockout.

Frame handling & battery

- `requestAnimationFrame` auto-throttles to the display and **pauses on hidden tabs** (saves battery).
- HUD/telemetry updates throttled to 200 ms; voice cues throttled (≥ 1.5 s gaps) to avoid spam and audio overlap.
- Lighter "Lightning" model + "lite" BlazePose variant chosen for mobile battery/thermals.

Render performance

- Per-frame numbers kept in refs; only user-visible values trigger React updates.
- Heavy TF.js trainer dynamically imported so other pages stay light.
- Service-worker cleanup avoids stale, slow cached bundles.

18. Debug System (Developer Mode)

A complete, **hidden** developer toolkit for tuning the AI — invisible to normal users (production always defaults to Auto/2D). Unlock once with `?unlock=forge` (persists in `localStorage["forge:dev"]`); always on in dev builds.

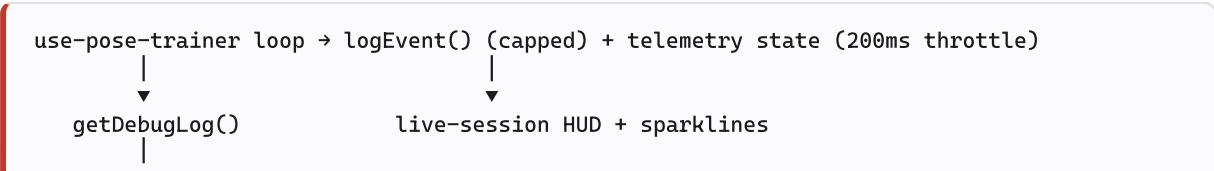
Components

Feature	What it does	Why it helps
Developer Settings	One-Euro sliders (minCutoff/beta/dCutoff), engine override (Auto/2D/3D), HUD toggle, device-tier readout	Tune smoothing & A/B engines without code changes
Live HUD	On-screen FPS, inference ms, engine, fallback, confidence, landmark count, rep state, exercise	See exactly what the AI sees in real time
FPS sparkline	Mini graph of FPS over time (ref line at 30)	Spot frame drops/thermal throttling
Inference sparkline	Mini graph of inference ms (ref 33, auto-max)	Compare model cost (2D vs 3D)
Debug log	Structured events (sample/rep/rep-rejected/tracking-loss/fallback), capped 8000	Reconstruct exactly what happened
JSON export	Full session + log dump	Deep offline analysis
CSV export	Per-set rows with full metric columns	Open in Excel/Sheets, pivot
Session tagging	Tag exercise/weight/angle/device/lighting/PR/failure/notes	Make comparisons meaningful
Session History dashboard	Auto-saved per-session records: search/filter/sort, favorite, delete, multi-select export, Compare (2D vs 3D side-by-side), Trends sparklines + top-error exercises	Longitudinal tuning & regression tracking

How it's kept hidden

- Dev pages render "not available" unless unlocked; nothing is linked in the normal navigation.
- Engine override + HUD live behind the dev gate; **production stays on Auto**.
- All telemetry is **device-local** (localStorage) — no server cost, no production schema, no user data exposure.

Telemetry data flow



▼ (on set/session end, dev only)
session-report → computeStats() → saveSession() → dev-history.ts (localStorage)
↓
/settings/developer/history → table · compare · trends · export

19. Security & Privacy

Camera permissions & privacy

- The camera only opens after an explicit **user gesture** (browser-enforced), and the browser shows its own in-use indicator.
- **Video never leaves the device** for inference — pose detection runs locally. Recordings are saved locally (download / Web Share); we don't upload media.
- This is the strongest privacy story possible for a camera app: the sensitive data (your body on video) stays on your hardware.

Authentication & session security

- Passwords hashed with **bcrypt** (salted, slow) — never stored in plaintext.
- Auth token is a signed **JWT** in an **HTTP-only, Secure, SameSite** cookie — unreadable by JS (XSS-resistant) and not sent cross-site (CSRF-resistant).
- Google sign-in tokens are verified **server-side** before we trust them.
- **Mandatory login:** `src/proxy.ts` blocks all app/trainer routes for unauthenticated users.

Data protection

- Input validated with **zod** on every API route (rejects malformed/malicious payloads).
- Prisma uses parameterized queries → **SQL-injection safe**.
- Secrets (DB URL, JWT secret, OAuth IDs) are **environment variables**, never committed; `.env` is gitignored.
- Only structured workout data is stored server-side — minimal personal data footprint.

Local storage scope

localStorage holds only non-sensitive device preferences and **dev-only** telemetry. No passwords or tokens are kept there.

Operational note: real database credentials were shared in chat during setup — those should be **rotated**. Always treat any secret that touches a chat/log as compromised.

Future cloud architecture (when added)

- If recordings ever sync to the cloud: encrypt at rest + in transit, signed time-limited upload URLs, per-user storage isolation, explicit opt-in consent.
- Rate-limiting + audit logging on APIs; secret manager instead of plain env vars at scale.

21. 100 Interview Questions & Answers

Grouped beginner → intermediate → advanced. Each: answer, why it's right, and the common mistake.

Beginner (1–30)

1. What is FORGE / AI-Trainer?

A web app that uses your camera + on-device AI to detect your body, count reps, score form, and coach you by voice in real time. *Why right:* it captures the three pillars (detect, count, coach). *Mistake:* calling it "just a workout logger" — the AI vision is the core.

2. What problem does it solve?

Affordable, always-available form correction + automatic tracking without a human trainer. *Mistake:* focusing only on counting and forgetting the injury-prevention/form angle.

3. What tech stack does it use?

Next.js 16 + React 19 + TypeScript (frontend/API), TensorFlow.js (on-device pose AI), PostgreSQL + Prisma (data), JWT auth. *Mistake:* saying the AI runs on a server — it runs in the browser.

4. Why run the AI in the browser, not the server?

Zero inference latency, near-zero server cost, and privacy (video never leaves the device). *Mistake:* assuming server GPUs are "more powerful so better" — the latency/cost/privacy win dominates for live coaching.

5. What is pose detection?

Locating body joints (landmarks) in an image, each as (x, y) + a confidence score. *Mistake:* confusing it with object detection (boxes) — pose gives joint points.

6. Which pose model is the default?

MoveNet MULTIPOSE_LIGHTNING (2D, multi-person, fast). *Mistake:* saying MediaPipe — that's used selectively via BlazePose and was stubbed at build.

7. How many landmarks does MoveNet return?

17 (nose, eyes, ears, shoulders, elbows, wrists, hips, knees, ankles). BlazePose returns 33. *Mistake:* mixing up the two counts.

8. What is a confidence score?

The model's certainty (0–1) that a joint is where it says. We gate logic on it. *Mistake:* treating all detected joints as equally trustworthy.

9. How is a rep counted at a high level?

We turn a joint angle into 0→1 "progress", then count when the user reverses out of a deep-enough bottom. *Mistake:* "we count when the leg straightens fully" — we count on the turnaround, not lockout.

10. What is the One-Euro filter for?

To remove landmark jitter without adding lag, so reps and form scores are stable. *Mistake:* calling it a simple average — it's adaptive to speed.

11. Why TypeScript?

Pose data is structured (`{x,y,score}`); types catch bugs before runtime. *Mistake:* "it's just preference" — it materially prevents a class of bugs here.

12. Why Next.js?

One codebase for UI, API, auth, and SSR with great routing/deploy. *Mistake:* thinking it's only a frontend framework.

13. What is the App Router?

Next.js's file-based routing where folders map to URLs and components render on the server by default. *Mistake:* confusing it with the older Pages Router.

14. What is a Server Component?

A component rendered on the server, shipping no JS and able to access secrets/DB directly. *Mistake:* using browser APIs in one without `"use client"`.

15. Why is the trainer a full-screen route outside the app shell?

The spec requires only camera + minimal overlays, no sidebar/chrome. *Mistake:* nesting it in the dashboard layout.

16. How does login work?

bcrypt-hashed password, then a signed JWT stored in an HTTP-only cookie. *Mistake:* storing the token in localStorage (XSS risk).

17. Why HTTP-only cookies for the token?

JavaScript can't read them, so XSS can't steal the session. *Mistake:* assuming any cookie is automatically secure.

18. What database and ORM?

PostgreSQL (Neon) via Prisma. *Mistake:* saying NoSQL — the data is relational (users→sessions→sets).

19. What is Prisma?

A type-safe ORM that generates a typed client from a schema and handles migrations. *Mistake:* calling it a database — it's the access layer.

20. Where are workout videos stored?

On the device (recorded via MediaRecorder, saved locally). *Mistake:* assuming they upload to the server.

21. What does the camera guide do?

Validates visibility/angle/lighting/framing and auto-starts the set when you're positioned. *Mistake:* thinking there's a manual Start button — it auto-starts.

22. What is "progress" in rep counting?

The joint angle normalized to 0 (top) → 1 (bottom). *Mistake:* confusing it with the user's training progress.

23. What's the difference between Beginner and Advanced mode?

Required depth eases +12° for beginners, tightens -6° for advanced; advanced also rejects bad-form reps. *Mistake:* thinking modes only change UI.

24. How does voice coaching work?

The Web Speech API speaks counts/cues in a simple-English persona, throttled to avoid spam. *Mistake:* expecting it to be recorded into the video (browsers can't capture system TTS).

25. What is body-lock?

Locking the rep count onto one person so passers-by don't hijack it. *Mistake:* assuming multi-person means it counts everyone.

26. What does the dashboard show?

Streak, volume, weekly chart, history — aggregated from saved sessions. *Mistake:* thinking it computes live during a workout.

27. How is the app deployed?

GitHub push to main → Render auto-builds (`prisma generate` + `next build`) → live HTTPS.

Mistake: forgetting the Prisma generate step.

28. Why Tailwind CSS?

Fast, consistent styling with a custom dark-luxury theme. *Mistake:* thinking it bloats the bundle — unused classes are purged.

29. What runs per frame?

Capture → detect → lock → filter → smooth → angle → rep → form → voice → draw, ~30×/sec. *Mistake:* putting DB writes in the loop (only at set/session end).

30. Is the exercise auto-detected?

No — the user picks it, and we load that exercise's tuning config. *Mistake:* claiming an ML classifier picks the exercise.

Intermediate (31–65)

31. Explain the rep state machine.

Two phases (UP/DOWN). UP→DOWN when progress exceeds ENTER and rises past the valley+TURN; in DOWN we track the peak; when progress drops below peak-TURN the rep is evaluated. *Mistake:* describing it as a simple threshold cross (that double-counts on jitter).

32. Why count on turnaround instead of lockout?

Gym testing showed lockout-based counting only fired when the user paused; turnaround counts mid-motion and handles heavy grind reps. *Mistake:* not citing the real-world failure that motivated it.

33. How do you prevent jitter double-counts?

One-Euro smoothing + a TURN reversal band + MIN_REP_PEAK depth gate. *Mistake:* relying on smoothing alone.

34. Give the One-Euro formula.

$\alpha = 1/(1 + \tau/\Delta t)$, $\tau = 1/(2\pi f_c)$; estimate speed $\hat{x}$, set $f_c = \text{minCutoff} + \beta \cdot |\hat{x}|$, then $\hat{x} = \alpha \cdot x + (1 - \alpha) \cdot \hat{x}_{\text{prev}}$. *Mistake:* forgetting the cutoff adapts to speed.

35. Why is One-Euro FPS-independent?

It uses real Δt timestamps, so behavior is the same at 24 or 60 FPS. *Mistake:* assuming a fixed per-frame coefficient.

36. What are minCutoff and beta?

minCutoff = baseline smoothing when still; beta = how much smoothing relaxes with speed. *Mistake:* swapping their roles.

37. How is a joint angle computed?

Dot product of the two vectors from the middle joint: $\theta = \arccos((\mathbf{BA} \cdot \mathbf{BC}) / (|\mathbf{BA}| |\mathbf{BC}|))$. *Mistake:* forgetting to clamp cos to $[-1, 1]$ (NaN risk).

38. How is range of motion (ROM) scored?

peak progress ÷ idealProgress, clamped, ×100. *Mistake:* using the required (mode) depth instead of the ideal angle.

39. How is stability scored?

$1 - (\text{hip-sway range} \div \text{torso length}) \times 1.2$, clamped, ×100. *Mistake:* measuring absolute pixels without normalizing by torso (breaks across distances).

40. How is form scored?

$100 - \text{errors} \times 30 - \text{warns} \times 12$, floored at 0. *Mistake:* treating warnings and errors the same.

41. How do you avoid false form alarms?

Three gates: view match, confidence $\geq$ HIGH_CONF, and persistence for SUSTAIN (6) frames. *Mistake:* firing a cue on a single bad frame.

42. What is the confidence gate's effect on faults?

A low-confidence "error" is demoted to a "warning" so it won't reject a rep. *Mistake:* letting an unsure model reject valid reps.

43. How does the overhead-press false-rep fix work?

requireWristAboveShoulder: a straight elbow only counts as worked if the wrist is above the shoulder, so arms-at-sides isn't a rep. *Mistake:* relying on elbow angle alone.

44. How does mode change required depth?

An ease term ($\pm$ degrees) shifts the active angle before converting to required progress. *Mistake:* hard-coding one depth for everyone.

45. How do you pick which body side to measure?

`bestSide` averages joint confidence per side and uses the more visible one. *Mistake:* always using the left side.

46. What is orientation detection for?

To know if the camera sees front or side, so form checks only run when valid. *Mistake:* running side-only checks on a front view.

47. Why refs instead of state in the loop?

Updating React state 30 $\times$ /sec re-renders the tree and tanks FPS; refs avoid that. *Mistake:* putting the rep count in state and writing it every frame.

48. How is the HUD kept cheap?

Telemetry is throttled to every 200 ms and only shown in dev. *Mistake:* updating the HUD every frame.

49. What is the hybrid 2D/3D pipeline?

`model-select.ts` picks 3D only for depth-sensitive exercises on capable devices; otherwise 2D. *Mistake:* using 3D everywhere (slower, single-person).

50. How does device-tier fallback work?

Static tier gates 3D; a runtime FPS EMA downgrades 3D $\rightarrow$ 2D if FPS stays below a floor $\sim$ 2.5 s (Auto only). *Mistake:* trusting the static check alone.

51. Why does 3D mode weaken body-lock?

BlazePose is single-person, so multi-person re-identification isn't available. *Mistake:* claiming 3D is strictly better.

52. How are reps validated vs rejected?

Reject if depth < required (incomplete) or, in advanced, a sustained high-confidence error; else valid. *Mistake:* rejecting on any momentary fault.

53. What's the occlusion-hold mechanism?

If a joint's score < 0.25, the smoother returns its last good position instead of a noisy jump. *Mistake:* letting an occluded joint snap to a wrong location.

54. How is the workout recorded?

`canvas.captureStream()` $\rightarrow$ MediaRecorder, capturing the skeleton overlay too. *Mistake:* recording the raw video element without overlays.

55. Why can't the voice be in the recording?

Web speechSynthesis output isn't routed to a capturable media stream. *Mistake:* promising in-video voice on the web (needs native).

56. How is input validated on the API?

zod schemas parse every request body; invalid input is rejected before the DB. *Mistake:* trusting client data.

57. How are SQL injections prevented?

Prisma uses parameterized queries; we never build raw SQL strings. *Mistake:* concatenating user input into queries.

58. How does Google OAuth work here?

The Google ID token is verified server-side, then we mint our own JWT. *Mistake:* trusting the client-sent token without verification.

59. What does `src/proxy.ts` do?

Acts as Next 16 middleware: checks the JWT cookie and redirects unauthenticated users — enforcing mandatory login. *Mistake:* protecting only some pages.

60. How is failure-set (reps-to-failure) handled?

Same counting core; the user presses "Finish set" and we log the real reps achieved. *Mistake:* needing a preset target.

61. How do you handle too-fast (momentum) reps?

If <450 ms since the last rep, cue "slow down, control it". *Mistake:* still scoring them as perfect.

62. What's stored per workout session?

Per-set reps, weight, ROM/form/stability/confidence scores, volume — for analytics and PRs. *Mistake:* storing only total reps.

63. How does progressive overload tracking work?

Weights per set are saved; history surfaces PRs and lets the rest screen suggest the next load. *Mistake:* ignoring per-set weight.

64. Why canvas drawing over DOM for the skeleton?

Canvas is GPU-composited and avoids layout thrash at 30 FPS. *Mistake:* drawing joints as DOM elements.

65. What's the role of `requestAnimationFrame`?

It drives the loop synced to the display and pauses on hidden tabs (battery). *Mistake:* using `setInterval` (drifts, runs in background).

Advanced (66–100)

66. Walk through the full per-frame pipeline.

rAF tick → read video frame → model inference (GPU) → body-lock select person → confidence filter + occlusion hold → One-Euro smoothing → bestSide + joint angle → progress → rep state machine → form checks → voice → canvas draw → (dev) telemetry. *Mistake:* omitting body-lock or putting smoothing after the angle.

67. Why a heuristic state machine instead of an ML rep classifier?

It's interpretable, needs no training data, runs in $O(1)$ /frame, and is tunable per exercise; an ML counter needs labeled data and adds latency. *Mistake:* assuming ML is automatically better.

68. How would you reduce false reps further without 3D?

Tighten TURN/MIN_REP_PEAK, add velocity-based rep validation, require minimum rep duration, and

cross-check a secondary joint. *Mistake*: only increasing smoothing (adds lag, hides real turnarounds).

69. Why is z in BlazePose only "estimated"?

It's regressed from a single 2D image by a model trained on 3D data — an inference, not a measured depth (no stereo/ToF). *Mistake*: treating z as metric-accurate.

70. Trade-offs of MoveNet Lightning vs Thunder?

Lightning: faster, multi-person, slightly less accurate. Thunder: more accurate, slower, single-pose. We need speed + multi-person for body-lock, so Lightning. *Mistake*: picking accuracy without weighing FPS/lock.

71. How does the FPS guard avoid oscillation (flapping)?

It downgrades only after FPS stays below the floor for a sustained ~2.5 s window (EMA), and doesn't auto-upgrade mid-session. *Mistake*: switching engines on a single slow frame.

72. What's the complexity of the whole per-frame pipeline?

Dominated by the CNN inference (fixed cost for a fixed input); the rest is $O(n)$ over ~17–33 landmarks, effectively constant. *Mistake*: claiming it scales with video resolution linearly in our code (the model fixes input size).

73. How do you keep memory stable over a long session?

Per-frame tensors are disposed by the detector, per-frame data lives in refs (no React garbage), and logs/history are capped (8000 / 200). *Mistake*: accumulating per-frame arrays in state.

74. How would you add real exercise auto-classification later?

Feed a window of normalized landmark sequences into a small temporal model (1D-CNN/LSTM/transformer) trained on labeled clips, with a confidence threshold + hysteresis before switching. *Mistake*: classifying per single frame (unstable).

75. How is the dev tooling kept out of production?

Gated by `NODE_ENV !== "production" || localStorage["forge:dev"] === "1"`; unlinked routes render "not available"; production forces Auto. *Mistake*: shipping a visible toggle.

76. Why localStorage for dev telemetry instead of a DB table?

Zero server cost, no production schema, no user-data exposure, instant. Trade-off: per-device, not synced. *Mistake*: adding a server table for dev-only data.

77. How does the Compare feature help tuning?

Side-by-side metrics for 2+ tagged sessions isolate one variable (2D vs 3D, device, angle, lighting, build) to see its effect on FPS/accuracy. *Mistake*: comparing runs where multiple variables changed.

78. What's your strategy for handling occlusion from equipment?

Confidence threshold to ignore, occlusion-hold to bridge brief gaps, bestSide to use the visible side, and camera-guide to position so key joints stay visible. *Mistake*: trusting a low-confidence joint.

79. How do you measure inference cost live?

Time the detect call, smooth it with an EMA, show it (and a sparkline) in the dev HUD; compare 2D vs 3D. *Mistake*: timing the whole frame (includes draw) and blaming the model.

80. Why throttle voice cues?

speechSynthesis queues; un-throttled cues overlap and annoy. We enforce ≥ 1.5 s gaps and dedupe per rep. *Mistake*: speaking every cue immediately.

81. How do you guarantee audio unlocks on mobile?

The first speech call must be inside a user gesture; we call `unlockVoice()` on Start/toggle and keep the engine alive. *Mistake*: trying to speak before any interaction (autoplay block).

82. How would you support offline use?

Cache the model + app shell in a service worker (carefully, given the stale-bundle bug), keep inference

local, and queue session saves for later sync. *Mistake*: a naive SW that serves stale JS (we hit this).

83. How do you handle the front-camera mirror in math?

Front view is visually mirrored for UX, but we flip coordinates back so left/right joints and angles stay correct. *Mistake*: letting the mirror swap detected sides.

84. What's the biggest accuracy limitation and how do you communicate it?

2D can't see depth-only faults (back rounding, heel lift) from a bad angle; we report those conservatively (warn, not reject) and the guide pushes a side view. *Mistake*: over-claiming perfect form detection.

85. How do you prevent a passer-by from stealing the count?

Body-lock selects one person near the lock center and re-identifies them each frame; the user can tap to set/reset the target. *Mistake*: always counting the highest-confidence person.

86. How would you scale the backend for many users?

It's already light (no inference server); scale the stateless Next service horizontally, use Neon pooling/read replicas, add caching + rate limiting. *Mistake*: worrying about GPU scaling (there is none server-side).

87. What are the costs of this architecture vs a server-AI one?

Near-zero marginal compute cost (client does AI) → high gross margin; server-AI would need GPUs per concurrent user. *Mistake*: ignoring unit economics.

88. How do you test the rep counter?

Replay recorded landmark sequences through `RepCounter.update()` and assert counts/scores; tune thresholds against slow/fast/heavy/partial cases. *Mistake*: only manual gym testing (not reproducible).

89. How is the camera-guide auto-start debounced?

All checks must stay green for a sustained period before a countdown, so a momentary good frame doesn't trigger. *Mistake*: starting on the first all-green frame.

90. Why EMA on top of One-Euro for the rep angle?

A light secondary smoothing (SMOOTH 0.35) further steadies the single scalar driving the state machine. *Mistake*: over-smoothing and erasing the turnaround.

91. How would you add injury-risk detection?

Combine sustained high-severity faults (e.g. valgus, lumbar flexion under load) with fatigue trend + load, flag risk, and auto-reduce intensity suggestions. Best with 3D/side view. *Mistake*: flagging single frames as injury.

92. How do you detect fatigue across a set?

Trend ROM, rep tempo, and form scores over reps; a declining curve near the end signals fatigue (shown in the rest debrief). *Mistake*: using a single rep's score.

93. What happens if the model fails to load?

The hook enters an error state with a user-facing message; the loop doesn't start; we can retry/fallback engine. *Mistake*: a blank screen with no feedback.

94. How do you keep the design system consistent?

Tailwind v4 theme tokens in `globals.css` + reusable `ui/` primitives (Button asChild, Switch, ProgressRing). *Mistake*: ad-hoc inline styles per page.

95. Why the `asChild` /Slot pattern on Button?

To render a link styled as a button without invalid `<a><button>` nesting (a real bug we fixed). *Mistake*: wrapping a button in an anchor.

96. How would you A/B test smoothing params with real users?

Bucket users, vary One-Euro params server-side flag, collect rep-accuracy/false-rate telemetry,

compare — exactly what the dev Compare/Trends prototype anticipates. *Mistake*: changing params globally with no measurement.

97. What's the data flow from a finished set to the chart?

RepCounter.summary() → set entry → POST /api/sessions → Prisma → PostgreSQL → dashboard query → Recharts. *Mistake*: computing charts on the client from in-memory state only.

98. How do you handle a dropped/loose tracking mid-rep?

Frames with sub-threshold joints are skipped (tracking=false), no phantom rep; a tracking-loss event is logged; the rep resumes when tracking returns. *Mistake*: counting through the gap.

99. What would break first under poor lighting, and your mitigation?

Landmark confidence drops → reps gate out and faults demote to warnings; the guide warns and asks for better light. *Mistake*: letting low-confidence data drive counts.

100. If you had to ship one accuracy upgrade next, what and why?

Velocity-aware rep validation + a proper offline replay test harness — biggest reduction in false/missed reps with no UX cost, and it makes tuning measurable. *Mistake*: jumping to 3D-everywhere (cost/lock regressions) before exhausting cheap 2D wins.

22. 100 Viva Questions & Answers

Short, exam-style Q&A for college presentations and vivas.

1. **What is the project's full name?** FORGE — an AI fitness trainer web app.
2. **What is its one-line purpose?** Watch you through the camera and coach your workout in real time.
3. **Name the core feature.** The AI Camera Trainer (pose-based rep counting + form coaching).
4. **What language is the codebase in?** TypeScript.
5. **Which framework?** Next.js 16 (App Router) with React 19.
6. **Which AI library?** TensorFlow.js.
7. **Default pose model?** MoveNet MULTIPOSE_LIGHTNING.
8. **3D pose model?** BlazePose (tfjs runtime).
9. **How many 2D landmarks?** 17.
10. **How many 3D landmarks?** 33.
11. **Where does inference run?** In the browser, on the device GPU.
12. **Why on-device?** Low latency, low cost, privacy.
13. **Database used?** PostgreSQL (Neon).
14. **ORM used?** Prisma.
15. **Auth method?** JWT in an HTTP-only cookie + bcrypt passwords.
16. **What is a landmark?** A detected body joint with (x, y) and a confidence score.
17. **What is a confidence score?** The model's certainty (0–1) about a joint's location.
18. **How is a joint angle measured?** Dot product of the two vectors at the middle joint.
19. **What is "progress"?** The joint angle normalized to 0 (top) → 1 (bottom).
20. **How is a rep counted?** On the bottom turnaround after reaching valid depth.
21. **What is the One-Euro filter?** An adaptive low-pass filter that removes jitter without lag.
22. **Why use it?** Stable angles → accurate reps and form scores.
23. **Is it FPS-independent?** Yes — it uses real timestamps (Δt).
24. **Name its main parameters.** minCutoff, beta, dCutoff.
25. **What does minCutoff control?** Baseline smoothing when nearly still.
26. **What does beta control?** How much smoothing relaxes with speed.
27. **What is occlusion hold?** Freezing a low-confidence joint at its last good position.
28. **What is body-lock?** Tracking one person so others don't affect the count.
29. **What is the rep state machine's two phases?** UP and DOWN.
30. **What confirms a direction change?** A reversal past the TURN band (0.1).
31. **What is MIN_REP_PEAK for?** Ignoring shallow fidget movements (must reach 0.5).

- 32. **How is ROM scored?** $\text{peak} \div \text{ideal progress} \times 100$.
- 33. **How is stability scored?** $1 - (\text{hip sway} \div \text{torso length}) \times 1.2, \times 100$.
- 34. **How is form scored?** $100 - \text{errors} \times 30 - \text{warns} \times 12$.
- 35. **When is a rep "perfect"?** $\text{ROM} \geq 90, \text{form} \geq 85, \text{stability} \geq 80$.
- 36. **When is a rep rejected?** Too shallow, or (advanced) a sustained confident form error.
- 37. **How many frames must a fault persist?** 6 (SUSTAIN).
- 38. **What are the three form gates?** View match, confidence, persistence.
- 39. **What is HIGH_CONF?** 0.5 — needed for an error to reject a rep.
- 40. **How does Beginner mode differ?** Depth target eased by $+12^\circ$.
- 41. **How does Advanced mode differ?** Stricter depth (-6°) and rejects bad-form reps.
- 42. **How is the exercise chosen?** The user selects it; we load its config (no auto-classification).
- 43. **What is poseKey?** The key linking an exercise to its rep configuration.
- 44. **Name a config family.** squatLike (hip-knee-ankle, $160 \rightarrow 100 \rightarrow 70^\circ$).
- 45. **What special gate do presses use?** requireWristAboveShoulder.
- 46. **Why that gate?** So arms-at-sides (straight elbow) aren't counted as a rep.
- 47. **What does the camera guide check?** Visibility, angle, framing, lighting.
- 48. **What happens when checks pass?** The set auto-starts (countdown), no Start button.
- 49. **Best camera angle for squats?** Side-on (to see depth).
- 50. **Why front camera is mirrored?** So it feels like a mirror (coords are un-mirrored in math).

- 51. **What is requestAnimationFrame?** A browser callback that drives the loop in sync with the display.
- 52. **Why not setInterval?** It drifts and runs in background tabs; rAF pauses when hidden.
- 53. **Why refs not state in the loop?** To avoid re-rendering 30×/sec and tanking FPS.
- 54. **How often is the dev HUD updated?** Throttled to every 200 ms.
- 55. **What backend runs inference?** None — it's all client-side.
- 56. **What is a Server Component?** A component rendered on the server with no client JS.
- 57. **Why is the trainer a separate full-screen route?** To show only camera + minimal overlays.
- 58. **What protects the routes?** `src/proxy.ts` (Next 16 middleware) checking the JWT cookie.
- 59. **Which API saves a workout?** `POST /api/sessions`.
- 60. **What validates API input?** zod schemas.
- 61. **How are passwords stored?** bcrypt-hashed (salted), never plaintext.
- 62. **Why HTTP-only cookies?** JavaScript can't read them (XSS-safe).
- 63. **How is Google login verified?** Server-side token verification, then our own JWT.
- 64. **Where are videos stored?** On the device (local save / share).

65. **How is the workout recorded?** `canvas.captureStream` → `MediaRecorder`.
66. **Can the voice be in the recording?** No — web TTS can't be captured.
67. **What API speaks cues?** Web Speech (`speechSynthesis`).
68. **Why throttle cues?** To avoid overlapping, spammy speech.
69. **What unlocks audio on mobile?** A user gesture (Start/toggle) calling `unlockVoice()`.
70. **What is the hybrid pipeline?** Choosing 2D vs 3D by exercise and device.
71. **What is device-tier?** Classifying the device as low/mid/high.
72. **What is the FPS fallback?** Auto-downgrading 3D→2D if FPS stays too low.
73. **Why does 3D weaken body-lock?** BlazePose is single-person.
74. **What does z represent?** Estimated depth (not measured).
75. **Which model is multi-person?** MoveNet.
76. **Why MoveNet Lightning over Thunder?** Faster + multi-person for body-lock.
77. **What is bestSide?** Choosing the more confidently tracked body side.
78. **What is orientation detection?** Detecting front vs side camera view.
79. **Why does orientation matter?** Form checks only run on a valid view.
80. **What is the stability metric based on?** Hip sway normalized by torso length.
81. **What is a hold exercise?** A timed posture (e.g. plank) scored on body line.
82. **What is the generic counter?** A hip-drop fallback counter for untuned moves.
83. **What is the failure-set mode?** Reps-to-failure; user finishes the set manually.
84. **What is progressive overload?** Increasing weight over time, tracked per set.
85. **What does the rest screen show?** A per-set debrief + next-set weight suggestion.
86. **What does the session report show?** Scores, video, PRs, save/export.
87. **What charts library is used?** Recharts.
88. **What animation library?** Framer Motion.
89. **What CSS framework?** Tailwind CSS v4.
90. **How many exercises are catalogued?** 120 (≈ 77 AI-tracked).
91. **Where is exercise data seeded from?** `prisma/data/exercises.ts`.
92. **What is Developer Mode?** A hidden toolkit for tuning the AI.
93. **How is it unlocked?** `?unlock=forge` (persisted) or dev builds.
94. **Name two dev tools.** Live HUD and Session History dashboard.
95. **Where is dev telemetry stored?** `localStorage` (device-local).
96. **What does Compare do?** Side-by-side metrics for multiple sessions.
97. **What does Trends show?** FPS/inference/accuracy over time + top-error exercises.
98. **Where is the app deployed?** Render (auto-deploy from GitHub main).
99. **What is the build command?** `prisma generate` + next build.

100. **What's the biggest limitation?** 2D can't see depth-only faults from a bad angle — mitigated by camera guidance and conservative warnings.

23. Future Improvements

Idea	What it adds	How it could be built
AI workout recommendations	Suggest the next workout from history/goals	Rules + later a recommender over session data
Personalized training plans	Adaptive multi-week programs	Progression model using volume/PR trends
Fatigue detection	Auto-stop/deload when form decays	Trend ROM/tempo/form across reps (prototype already in rest debrief)
Injury-risk detection	Flag dangerous patterns (valgus, lumbar flexion)	3D/side-view + sustained high-severity faults + load
Nutrition integration	Calories/macros alongside training	Food DB + goal-based targets
Wearable integration	Heart rate, real calories, recovery	Web Bluetooth / Health APIs
Cloud analytics	Cross-device history + dashboards	Authenticated dev/user telemetry tables
LLM-powered coaching	Natural-language plans & explanations	Claude API summarizing session data into advice
Voice conversations	Ask the coach questions hands-free	Speech-to-text + LLM + TTS loop
Smartwatch support	Glanceable reps/rest on the wrist	Companion watch app
Native Android (Capacitor)	True gallery save + voice baked into recordings	Wrap the web app; use native media APIs
Auto exercise classification	Remove the "pick exercise" step	Temporal model (1D-CNN/LSTM) over landmark sequences
Automated test harness	Reproducible rep-accuracy regression tests	Replay recorded landmark sequences through RepCounter

24. Final Summary

Key takeaways

- FORGE is a **client-heavy, on-device AI** fitness coach: the browser does pose detection, rep counting, form analysis, and voice; the server only stores structured data.
- That single choice drives everything good about it: **low latency, low cost, strong privacy**.
- The AI is built from **interpretable, tunable algorithms** (state machine + geometry + One-Euro), not a black box — which makes it debuggable and reliable.
- Robustness comes from layered defenses: confidence gates, persistence gates, body-lock, occlusion-hold, and a runtime FPS fallback.

Technologies used

Next.js 16 React 19 TypeScript Tailwind v4 Framer Motion Recharts TensorFlow.js (WebGL) Prisma
PostgreSQL (Neon) jose JWT bcryptjs zod Web Speech API MediaRecorder getUserMedia Render

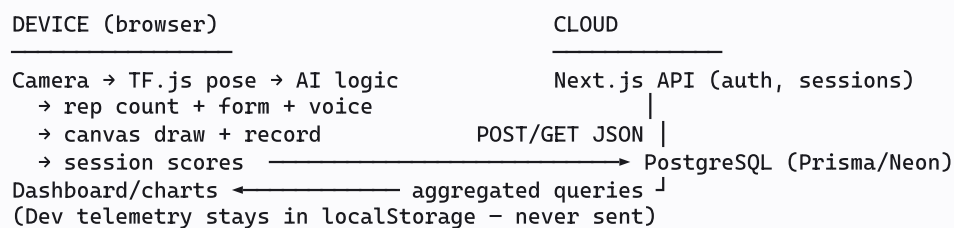
AI models used

MoveNet MULTIPOSE_LIGHTNING (2D) BlazePose (3D) Web speechSynthesis (TTS)

Algorithms used

CNN pose estimation One-Euro filter Bottom-turnaround state machine Dot-product joint angles EMA smoothing
Occlusion hold Body-lock tracking Device-tier + FPS fallback Hybrid 2D/3D selection Confidence filtering
Run-length form gating Sway stability metric Orientation detection canvas→MediaRecorder encoding

System architecture summary



Learning outcomes

- Real-time computer vision in the browser (TF.js, WebGL, pose estimation).
- Signal processing for noisy sensor data (One-Euro, EMA, gating).
- Designing interpretable algorithms (state machines, geometry) over ML black boxes.
- Full-stack engineering: typed APIs, relational data, secure auth, deployment.
- Performance engineering for a 30 FPS real-time loop (refs, canvas, adaptive models).
- Product thinking: honest limitations, privacy-by-design, and hidden developer tooling.

